

sans I/O

networking libraries without I/O

Apoorv Kothari



@toidiu.com

Personal Intro

- **Currently @Cloudflare working on Protocols**
- **Past: maintained QUIC and TLS libraries**

What is sans I/O

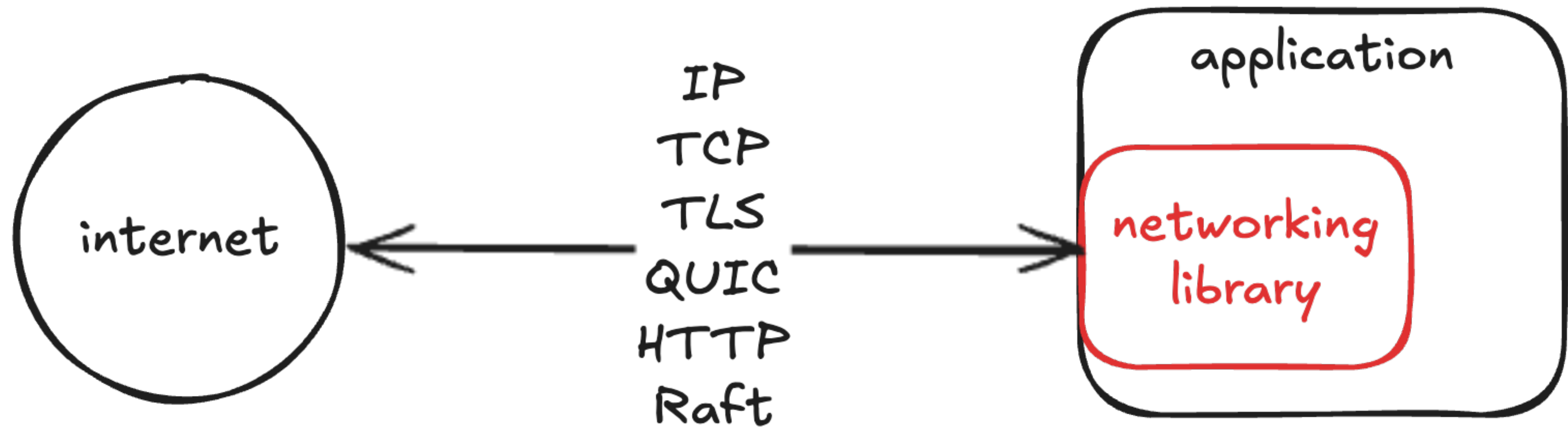
- *sans* == without.
 - Write your networking library “without I/O”
- Is that useful??

What you will learn

- Why do we want to avoid IO?
- What is sans IO?
- A word of caution about async
- Architecting a sans IO library

Why do we want to avoid IO?

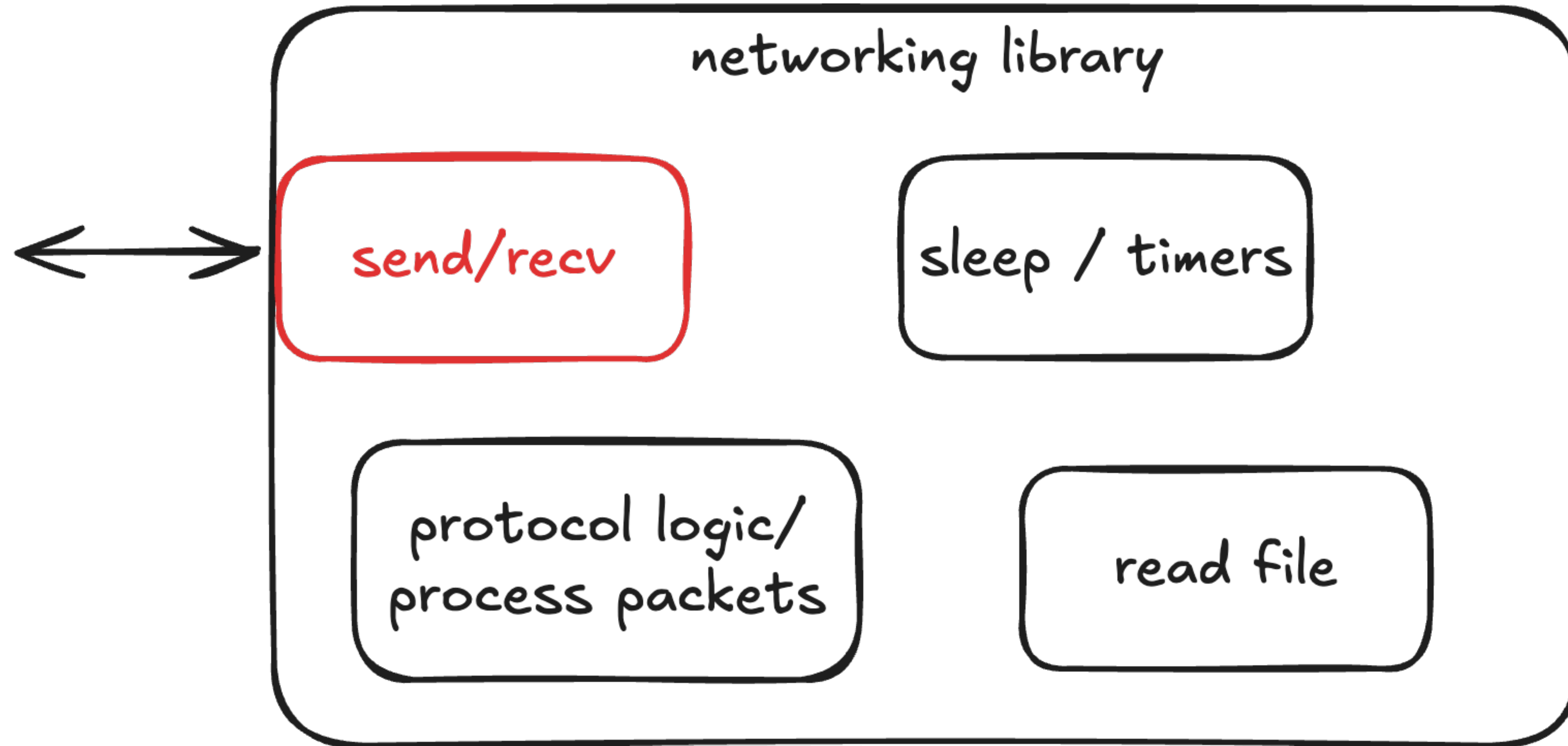
What is a networking library



- The rules and language use by devices to speak to each other.

The I/O component

aka. “async”



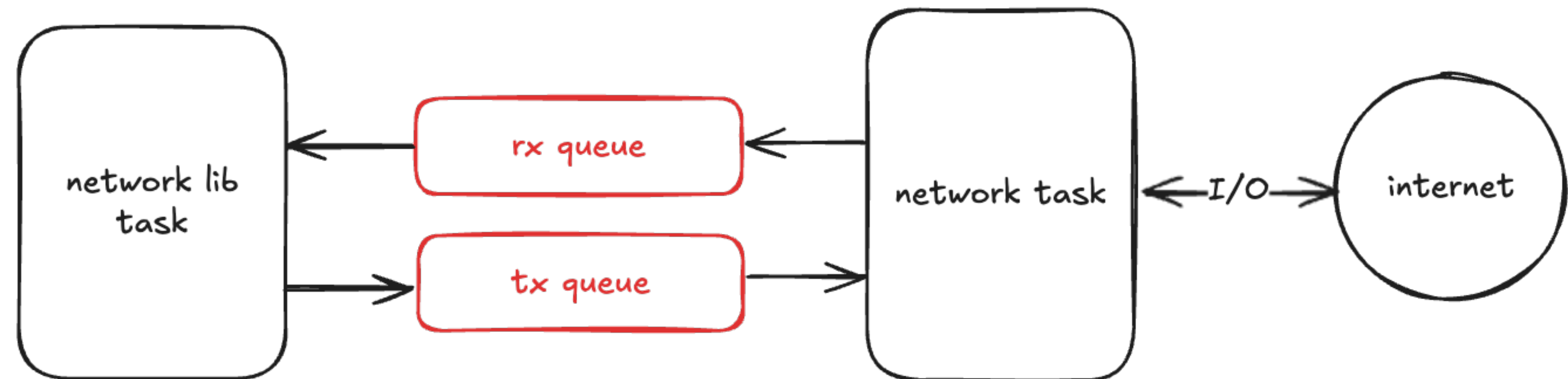
Why is I/O difficult

- **Socket APIs can fail:** are you handling all error conditions?
 - Process limit, Memory limit, Buffer full, Permissions, Blocked, Connection Reset by peer, System interrupts, actual application behavior etc.
- **APIs can be platform dependent**
 - Requires feature probing and branching
- **High-performance I/O is complicated**
 - Might require architecting your library
- **I/O requires non-blocking code. (aka. `async`)**
 - async adds complexity and is more difficult to reason about

What is sans I/O

Let's define sans I/O

- *sans* == without.
 - Write your networking library “without I/O”
- Is that useful??



Basic idea:

- Use **buffers** to abstract away I/O.

The promise of sans I/O

- **A well tested codebase.**
- Less complex and easier to maintain.
- Better performance!
- Portable across platforms.

Testing I/O

Testing a Raft protocol

```
#[tokio::test]
async fn test_start_election() {
    let mut peers = Peer::mock(&[1, 2, 3]);
    let mut candidate = Candidate::default(&mut peers);

    // Trigger election
    let transition = candidate.start_election(&mut context);

    // Expect RequestVote RPC sent to all peers
    let rpc_request_vote = Rpc::new_request_vote(state.current_term);

    for peer_io in peers.iter() {
        let rpc_sent = peer_io.send_queue.pop().unwrap();
        assert_eq!(rpc_request_vote, rpc_sent);
    }
}
```

Testing I/O

Testing a Raft protocol

```
#[tokio::test]
async fn test_start_election() {
    let mut peers = Peer::mock(&[1, 2, 3]);
    let mut candidate = Candidate::default(&mut peers);

    // Trigger election
    let transition = candidate.start_election(&mut context);

    // Expect RequestVote RPC sent to all peers
    let rpc_request_vote = Rpc::new_request_vote(state.current_term);

    for peer_io in peers.iter() {
        let rpc_sent = peer_io.send_queue.pop().unwrap();
        assert_eq!(rpc_request_vote, rpc_sent);
    }
}
```

A word of caution about async

Receiving an RPC

Implementing the Raft protocol

```
struct Server {  
    mode: Mode,  
    ...  
}  
  
pub enum Mode {  
    F(Follower),  
    C(Candidate),  
    L(Leader),  
}
```

Receiving an RPC cont...

Implementing the Raft protocol

```
impl Server {
    fn on_recv(&mut self, rpc: Rpc) {
        server.mode.on_recv(rpc);
    }
}

impl Mode {
    fn on_recv(&mut self, rpc: Rpc) {
        match self {
            Mode::F(follower) => follower.on_recv(rpc);
            ...,
        };
    }
}

impl Follower {
    fn on_recv(&mut self, rpc: crate::rpc::Rpc) {
        match rpc {
            Rpc::AE(append_entries) => self.on_recv_append_entries(append_entries),
            ...,
        }
    }
}
```


Receiving an RPC cont...

Implementing the Raft protocol

```
fn on_rcv_append_entries(&mut self, append_entries: AppendEntries, context: &mut Context) {  
    ...  
  
    // Prepare RPC  
    let leader_io = context.get_leader_io(&append_entries);  
    let mut buffer = vec![0; IO_BUF_LEN];  
    Rpc::new_append_entry_resp(current_term, response).encode_mut(&mut buffer);  
  
    leader_io.send(buffer.as_mut_slice());  
}
```

Receiving an RPC cont...

Implementing the Raft protocol

```
async fn on_rcv_append_entries(&mut self, append_entries: AppendEntries, context: &mut Context) {  
    ...  
  
    // Prepare RPC  
    let leader_io = context.get_leader_io(&append_entries);  
    let mut buffer = vec![0; IO_BUF_LEN];  
    Rpc::new_append_entry_resp(current_term, response).encode_mut(&mut buffer);  
  
    leader_io.send(buffer.as_mut_slice()).await;;  
}
```

Receiving an RPC cont...

Implementing the Raft protocol

```
impl Server {
    fn on_recv(&mut self, rpc: Rpc) {
        server.mode.on_recv(rpc);
    }
}

impl Mode {
    fn on_recv(&mut self, rpc: Rpc) {
        ...

        Mode::F(follower) => follower.on_recv(rpc);
    }
}

impl Follower {
    fn on_recv(&mut self, rpc: crate::rpc::Rpc) {
        ...

        Rpc::AE(append_entries) => self.on_recv_append_entries(append_entries),
    }
}

fn on_recv_append_entries(&mut self, append_entries: AppendEntries, context: &mut Context) {
    ...

    leader_io.send(buffer.as_mut_slice());
}
```


Receiving an RPC cont...

Implementing the Raft protocol

```
impl Server {
    async fn on_recv(&mut self, rpc: Rpc) {
        server.mode.on_recv(rpc).await;
    }
}

impl Mode {
    async fn on_recv(&mut self, rpc: Rpc) {
        ...

        Mode::F(follower) => follower.on_recv(rpc).await;
    }
}

impl Follower {
    async fn on_recv(&mut self, rpc: crate::rpc::Rpc) {
        ...

        Rpc::AE(append_entries) => self.on_recv_append_entries(append_entries).await,
    }
}

async fn on_recv_append_entries(&mut self, append_entries: AppendEntries, context: &mut Context) {
    ...

    leader_io.send(buffer.as_mut_slice()).await;
}
```

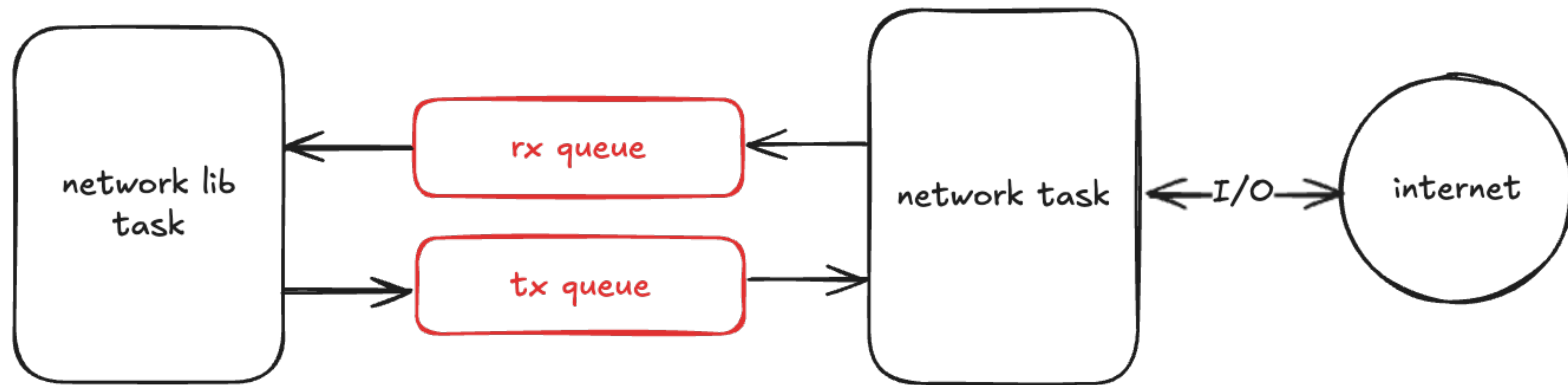
Before you reach for async

- **Less debuggable:** there is no linear stack
 - tokio console can help (one more tool and it's still not perfect).
- **More complexity:** more machinery
 - Library needs to care about Send, Sync, Pin, 'static.
- **New class of errors:** are you testing all concurrent flows?
 - Will my program block? deadlock?

Architecting a sans I/O library

Guidelines

- Avoid socket APIs and use buffers instead.
- Avoid async and manage Futures ourselves.



Reference: <https://www.qovery.com/blog/common-mistakes-with-rust-async>

Event loop

```
struct EventLoop {
    network: NetworkTask,
    server: Server,
}

impl EventLoop {
    async fn start(&mut self) {
        loop {
            // Receive/Transmit on the socket.
            self.network.rx().await;
            self.network.tx().await;

            // Poll and check if any timers have expired.
            self.server.timer().await;

            // Process received RPCs and buffer new RPCs to be sent.
            self.server.rx().await;
            self.server.tx().await;
        }
    }
}
```


Event loop

```
struct EventLoop {
    network: NetworkTask,
    server: Server,
}

impl EventLoop {
    async fn start(&mut self) {
        loop {
            // Receive/Transmit on the socket.
            self.network.rx().await;
            self.network.tx().await;

            // Poll and check if any timers have expired.
            self.server.timer().await;

            // Process received RPCs and buffer new RPCs to be sent.
            self.server.rx().await;
            self.server.tx().await;
        }
    }
}
```

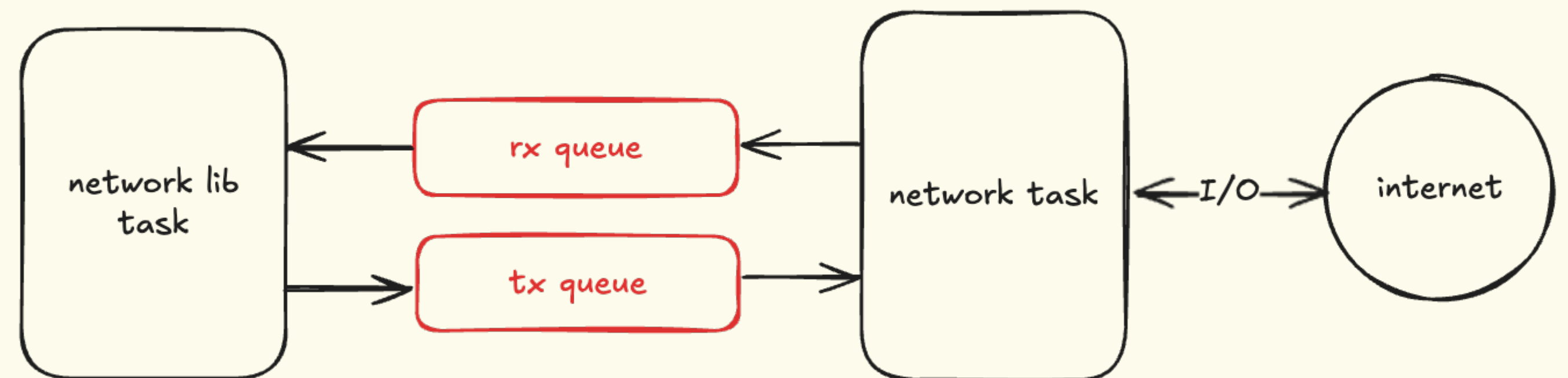
I/O buffers

```
impl BufferIo {
    pub fn split() -> (ServerIo, NetworkIo) {
        let rx_queue = Arc::new(Mutex::new(VecDeque::new()));
        let tx_queue = Arc::new(Mutex::new(VecDeque::new()));
        let rx_waker = Arc::new(Mutex::new(waker));
        let tx_waker = Arc::new(Mutex::new(waker));

        let server_io = ServerIo {
            buf: [0; IO_BUF_LEN],
            rx: rx_queue.clone(),
            tx: tx_queue.clone(),
            rx_waker: rx_waker.clone(),
            tx_waker: tx_waker.clone(),
        };

        let network_io = NetworkIo {
            ...
        };
        (server_io, network_io)
    }
}
```

```
/// A handle to the underlying BufferIo
#[derive(Debug)]
pub struct ServerIo {
    pub buf: [u8; IO_BUF_LEN],
    pub rx: Arc<Mutex<VecDeque<u8>>>,
    pub tx: Arc<Mutex<VecDeque<u8>>>,
    pub rx_waker: Arc<Mutex<Waker>>,
    pub tx_waker: Arc<Mutex<Waker>>,
}
```



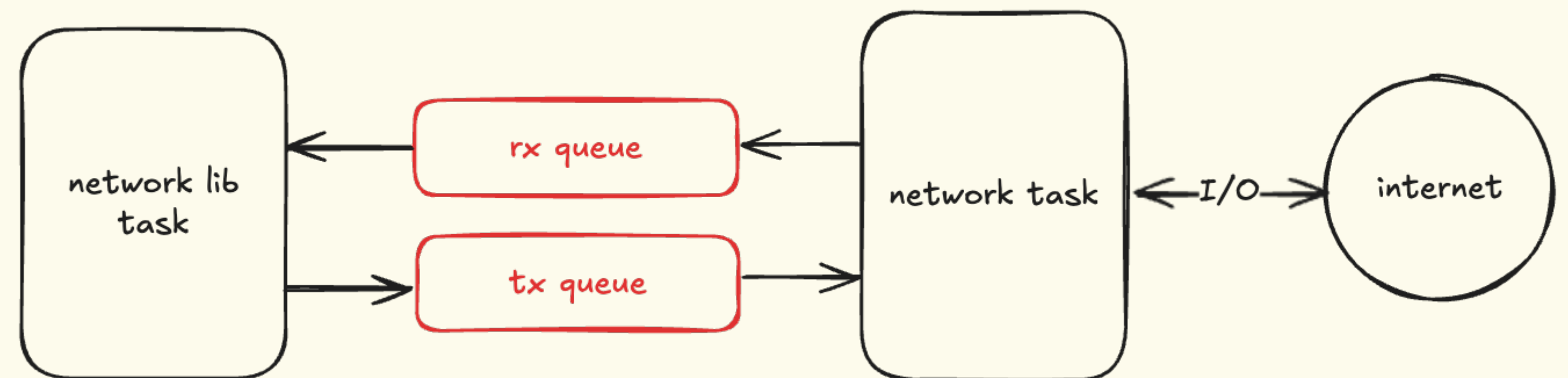
I/O buffers

```
impl BufferIo {
    pub fn split() -> (ServerIo, NetworkIo) {
        let rx_queue = Arc::new(Mutex::new(VecDeque::new()));
        let tx_queue = Arc::new(Mutex::new(VecDeque::new()));
        let rx_waker = Arc::new(Mutex::new(waker));
        let tx_waker = Arc::new(Mutex::new(waker));

        let server_io = ServerIo {
            buf: [0; IO_BUF_LEN],
            rx: rx_queue.clone(),
            tx: tx_queue.clone(),
            rx_waker: rx_waker.clone(),
            tx_waker: tx_waker.clone(),
        };

        let network_io = NetworkIo {
            ...
        };
        (server_io, network_io)
    }
}
```

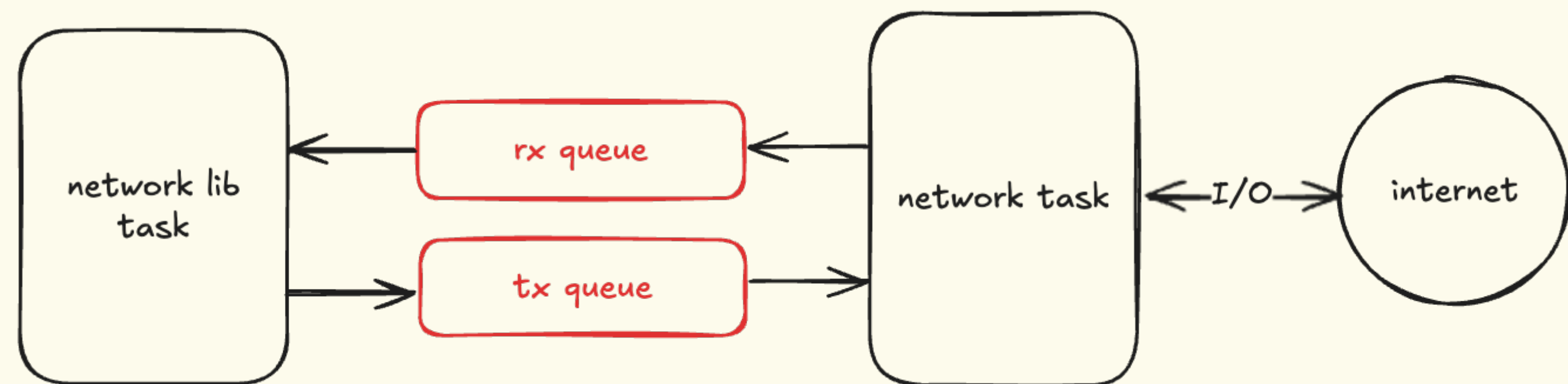
```
/// A handle to the underlying BufferIo
#[derive(Debug)]
pub struct ServerIo {
    pub buf: [u8; IO_BUF_LEN],
    pub rx: Arc<Mutex<VecDeque<u8>>>,
    pub tx: Arc<Mutex<VecDeque<u8>>>,
    pub rx_waker: Arc<Mutex<Waker>>,
    pub tx_waker: Arc<Mutex<Waker>>,
}
```



I/O buffers cont...

```
pub trait ServerTx {  
    fn send(&mut self, data: Vec<u8>);  
}
```

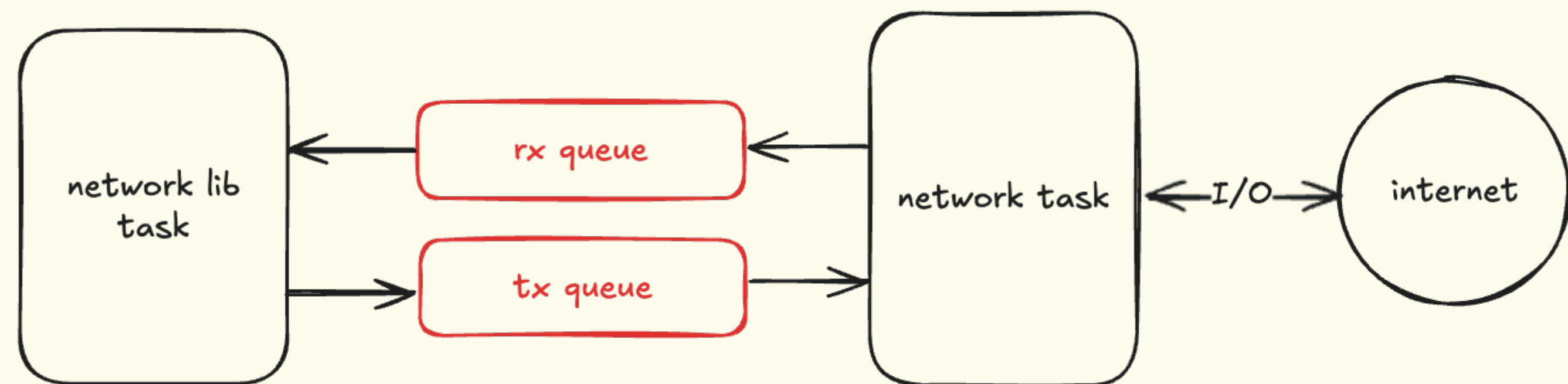
```
impl ServerTx for ServerIo {  
    fn send(&mut self, data: Vec<u8>) {  
        self.tx_queue.lock().unwrap().extend(data);  
  
        // Notify the network task that there is new data to send  
        self.tx_waker.lock().unwrap().wake_by_ref();  
    }  
}
```



I/O buffers cont...

```
pub trait ServerTx {  
    fn send(&mut self, data: Vec<u8>);  
}
```

```
impl ServerTx for ServerIo {  
    fn send(&mut self, data: Vec<u8>) {  
        self.tx_queue.lock().unwrap().extend(data);  
  
        // Notify the network task that there is new data to send  
        self.tx_waker.lock().unwrap().wake_by_ref();  
    }  
}
```



I/O buffers cont...

```
pub trait ServerRx {
    fn recv(&mut self) -> Option<Vec<u8>>;

    fn poll_rx_ready(&mut self, cx: &mut Context) -> Poll<()>;
}

impl ServerRx for ServerIo {
    fn recv(&mut self) -> Option<Vec<u8>> {
        self.rx_queue.lock().unwrap().read(&mut self.buf[0..]).ok()
    }

    fn poll_rx_ready(&mut self, cx: &mut Context) -> Poll<()> {
        let has_data = !self.rx_queue.lock().unwrap().is_empty();
        if has_data {
            Poll::Ready(())
        } else {
            Poll::Pending
        }
    }
}
```


I/O buffers cont...

```
pub trait ServerRx {
    fn recv(&mut self) -> Option<Vec<u8>>;

    fn poll_rx_ready(&mut self, cx: &mut Context) -> Poll<()>;
}

impl ServerRx for ServerIo {
    fn recv(&mut self) -> Option<Vec<u8>> {
        self.rx_queue.lock().unwrap().read(&mut self.buf[0..]).ok()
    }

    fn poll_rx_ready(&mut self, cx: &mut Context) -> Poll<()> {
        let has_data = !self.rx_queue.lock().unwrap().is_empty();
        if has_data {
            Poll::Ready(())
        } else {
            Poll::Pending
        }
    }
}
```

I/O buffers cont...

```
pub trait ServerRx {
    fn recv(&mut self) -> Option<Vec<u8>>;

    fn poll_rx_ready(&mut self, cx: &mut Context) -> Poll<()>;
}

impl ServerRx for ServerIo {
    fn recv(&mut self) -> Option<Vec<u8>> {
        self.rx_queue.lock().unwrap().read(&mut self.buf[0..]).ok()
    }

    fn poll_rx_ready(&mut self, cx: &mut Context) -> Poll<()> {
        let has_data = !self.rx_queue.lock().unwrap().is_empty();
        if has_data {
            Poll::Ready(())
        } else {
            Poll::Pending
        }
    }
}
```


References

- <https://sans-io.readthedocs.io/>
- <https://en.wiktionary.org/wiki/sans>
- <https://journal.stuffwithstuff.com/2015/02/01/what-color-is-your-function/>
- <https://www.firezone.dev/blog/sans-io>
- <https://www.qovery.com/blog/common-mistakes-with-rust-async>
- <https://man7.org/linux/man-pages/man2/send.2.html>

Thank you. Question.

